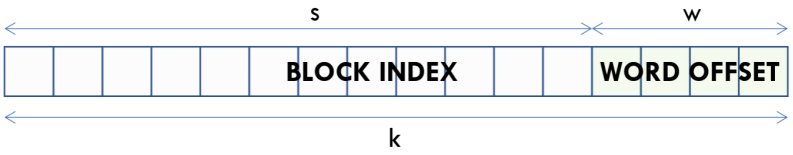
1 Lesson 14-10-26

In the last lesson we started cache memories.

1.0.1 Cache parameters

Let's first look at some *parameters* regarding cachelines and DRAM blocks.



From the figure we see how a standard address is divided in multiple regions, that is:

- The **block index**, of width s, which identifies the block the address pertains to;
- The **word offset**, of width w, which identifies a single word in the block (shared by multiple words).

  The parameters (s and w, as well as the full address size k) are defined below.

- **Mainsize**: the size of DRAM main memory (in words). We have:

$$k = \log_2(\text{Mainsize})$$

- **Cachesize**: the size of SRAM (in words);
- **Linesize**: the number of words in a cache line. We have:

$$w = \log_2(\text{Linesize})$$

- **Nlines**: the number of cache lines. It's defined as:

$$\text{Nlines} = \frac{\text{Cachesize}}{\text{Linesize}}$$

  We also define:

$$r = \log_2(\text{Nlines})$$

- **Nblocks**: the number of blocks in DRAM. It's defined as:

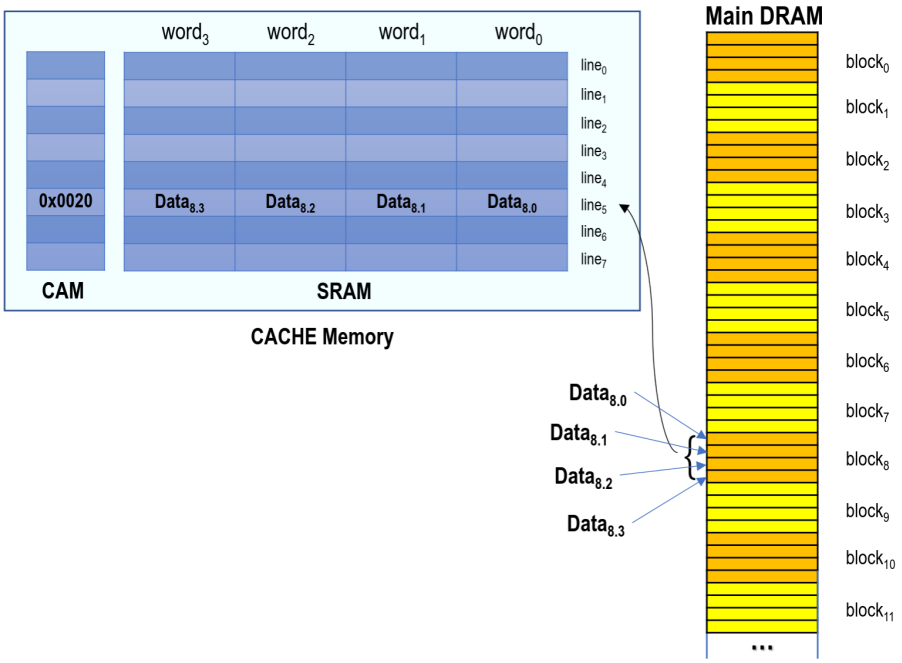
$$\text{Nblocks} = \text{Mainsize}Linesize$$

  We also define:

$$s = \log_2(\text{Nblocks})$$

1.0.2 CAM cache

The first approach introduced involved a CAM. Let's give a quick demonstration.



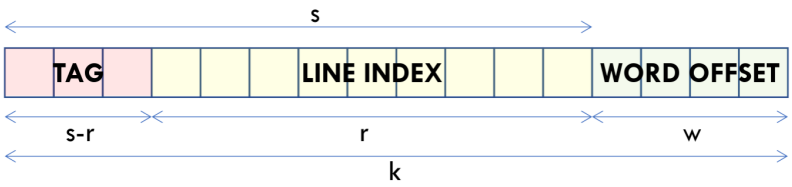
- Say the cache line is 4 words large, with the whole cache initially empty. The processor issues a read for the word at address `0x0022` (the Read Address). This belongs to the ninth block (block8), and the block address is `0x0020` (the Block Address). Say we mean to store block8 in the sixth cache line (line5): to do this we store its address in the sixth location of the CAM. After this is done, we return word2 from the cache line.
- Next, assume the processor issues a read for the word at address `0x0023` (the Read Address). This belongs to the ninth block (block8), as well, so when we match the Block Address `0x0020` to the CAM, we get a HIT, and the CAM returns the line5. At this point we can quickly read word3 from the cache line

There are some clear *policies* to define for this:

- When reading from DRAM and storing in cache, we must decide where to write the data. This is the **mapping** policy;
- If the cache is full and new data has to be written, some previous data must be erased. This is the **replacement** policy;
- When writing some data, cache and main memory should be both updated. this is the **consistency** policy.

1.0.3 Direct mapping cache

An alternative approach is one where the association between a block in main memory and a cache line is unique by design. Such an unique mapping, in hardware, is easily implemented via a modulo hash operation, by taking the least significant r bits of the block index as a line index, and the most significant bits as a unique tag.



In practice, we don't need to compute anything, as the index can be directly obtained from the address word by proper masking and/or shifting:

$$\text{Line} = (\text{Address} >> 2)\%(\text{Nlines} - 1)$$

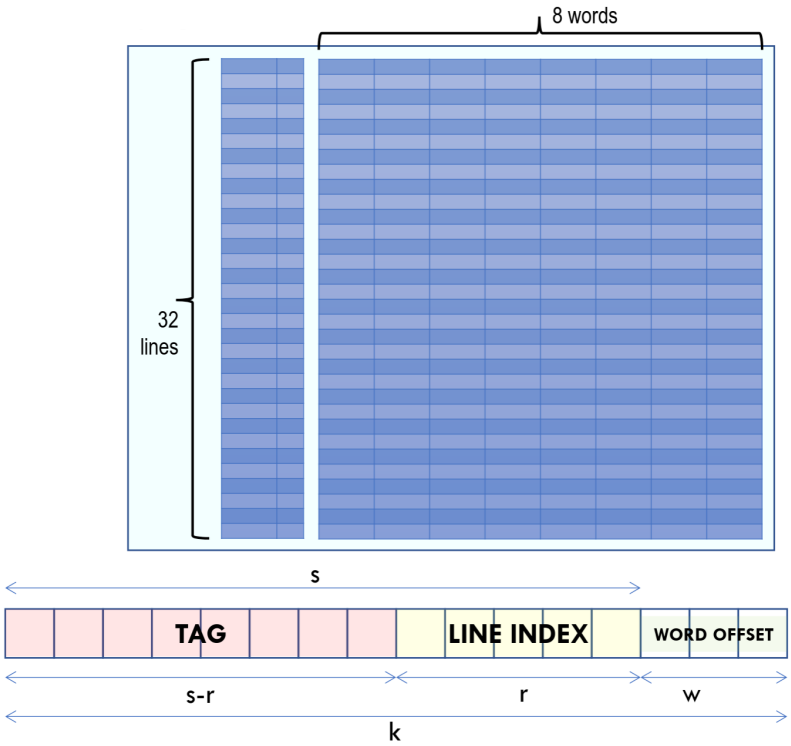
in binary, where $(\text{Address} >> 2)$ is nothing more than the *Block Address*.

Multiple blocks are assigned to the same cache line (as we said, the DRAM is bigger than the whole SRAM), with only $\frac{\text{Nblocks}}{\text{Nlines}}$ blocks mapped to each cache line. Only one of them is loaded in cache at any given time (so we have to *evict* old ones).

Consider blocks in main memory mapped to the same cache line.

- The address of each block always has the w least significant bits at 0, so we do not need to store this information in the CAM.
- The *Line* index is the same for all blocks assigned to the same cache line, as it corresponds to the cache line to which they are mapped, so we also do not need to store this information in the CAM.

Only the *TAG*s differ: the TAG is made of the upper $s - r$ bits of the address. This is the only information that needs to be stored in the CAM: different blocks mapped to the same cache line are guaranteed to have different TAG.



At this point, given the address of some data, what we need to do is:

- Match the TAG with the CAM at location pointed by *Line* index (only if the entry is valid). The CAM is actually made of *Nlines* separate CAMs of a single word;
- On MISS, we read the memory block from DRAM at address *Blockaddress*. The TAG is stored in the CAM, in the location pointed by *Lineindex*, so we can instantly match (and rewrite for misses). The block data is stored in the SRAM, in the line pointed by the *Line* index, possibly replacing other data already present. Then, a validity bit is set for the line, and we can read the word at *Word Offset* in the line pointed by the *Line* index.

A word can be spent on the validity bit: this is concatenated to the TAG, and when we match we match TAGs with the validity bit at 1. If the cacheline was never written to then the validity bit is at 0 and we definitely don't match;

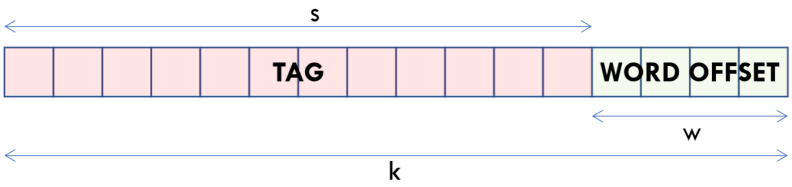
- On HIT, we just read the word at *Word Offset* in the line pointed by the *Line* index.

Let's look at the *performance* of this. The advantages are that direct mapping cache is very simple to implement, cheap. We don't even need a CAM, as the *Line* index already points to the only location to match. This makes for low power consumption, and the simplest replacement policy: no policy at all, since it's built into the memory structure.

The disadvantages are that we may replace some data even if the cache is not full (on misses), leading to poorer performance. The HIT Rate is also the poorest out of all cache designs we'll see.

1.0.4 Fully associative mapping cache

This is the one we introduced in 9.2.4. No pre-determined association is defined between a main memory block and a line: when loaded from DRAM, a block is stored in the first available cache line. This means that there is no pre-determined *Line* index.



Basically, in mapping terms, we can say that TAG is the same size as *Block* index, and there is no *Line* index.

At this point, given the address of some data, we match the TAG with the entire CAM (only valid entries).

- We first match the TAG with the CAM;
- On MISS, we read the memory block from DRAM at address *Blockaddress*. Need a replacement policy if cache is full. After that, the block data is stored in the SRAM, in the same line as TAG. A validity bit is set, and we read the word at *Word Offset* of the line;
- On HIT we read the word at *Word Offset* in the line that generated high when matching with the tag.

Let's talk about performance. The advantages are the best use of the cache, and the best hit rate possible. This means we ossibly get the best performance.

The disadvantages, of course, are the complexity to implement, the costs, and the fact that we need a large CAM with higher power dissipation and higher delays.

1.0.5 Set associative mapping cache

Set associative mapping is an in-between approach between direct mapping and fully associative.

Cache lines are grouped into **sets**:

- **Nsets** is the number of sets;
- **Setlines** the number of lines in a set.

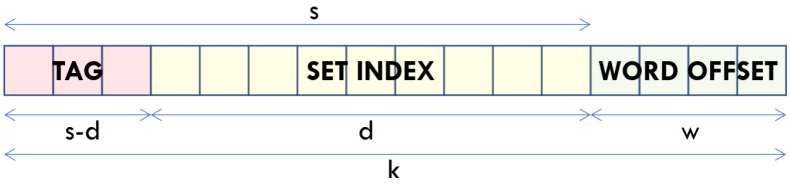
From this we have that:

$$\text{Nsets} = \frac{\text{Nlines}}{\text{Setlines}}$$

And the bits needed to identify a set are:

$$d = \log_2(\text{Nsets})$$

We have *direct association* of a main memory block towards a set, and *fully Associative association* within that set.



As with common direct mapping, the set index is Set $=$ Blockindex$\%$Nsets, and the TAG is $(s - d)$ bits long.

At this point, fiven the address of some data, we:

- Match the TAG with the CAM of the *Set* pointed by the *Set* index (only valid entries). There are *Nsets* separate CAMs, each of *Setlines* entries;
- On MISS, we Read the memory block from DRAM at address *Blockaddress*. The TAG is stored in the CAM of the *Set*, in the next empty line. We of course need a replacement policy if the set is full (**LRU**, *Least Recently Used*, is a good candidate). The block data is then stored in the SRAM, in the same line as TAG. A validity bit is set for the line, and we read the word at *Word Offset* in the line.
- On HIT, we read the word at *Word Offset* in the line that generated HIT when matching.

1.0.6 Cache replacement policies

When a cache is full, a cache line must be replaced to store a new one. That is our *cache replacement policy*. We have a few options:

- **Direct mapping**: we have no choice, may replace a cache line even if the cache is not full;
- **Associative mapping**: set associative mapping may replace a cache line even if the cache is not full (but the target set is full). The algorithms are:
  - **LRU** (*Least Recently Used*): we store timestamps of last usages. The con is the need to sort timestamps (within a set or within the full cache, for Set and Fully associative, respectively). Intel processors used some interesting heuristics to approximate this in the 486 era;
  - **LFU** (*Least Frequently Used*) we use counters to know the number of accesses. We need to sort number of accesses (within a set or within the full cache, for Set and Fully associative, respectively);
  - **FIFO** (*First In, First Out*): we restart from first line when the bottom (of a set or the full cache) is reached. A simple pointer is required (one for each set, or one for the full cache, for Set and Fully associative, respectively);
  - **Random** choice is not that useful, but sometimes involved in lower steps of more complex algorithms.

1.0.7 Cache consistency policies

We then need policies on how we handle *writes* thorugh the cache. That is, when writing, we have decide how to update cache and main memory. This is important especially in multi-core systems and with peripherals accessing the memory, where each core has a dedicated cache.

A few approaches:

- **Write through**: all writes go to main memory as well as cache. Multiple CPUs can monitor main memory traffic to keep local caches up to date When a core makes a write, the other cores invalidate the cache entry relative to the written block. This involves lots of traffic, which slows down write operation. On the other hand it has the best consistency. Plus note that on average, only 15% of memory references are writes.
- **Write fatback** the update is initially made in cache only. An update bit is then set for cache line (usually called the *dirty bit*). If line is to be replaced in cache, we write back to main memory if dirty bit is set. This saves a lot of traffic on the memory bus, and makes for improved performance. The tradeoff is of course the fact that other caches get out of sync with the dirty ones (or in fact they might have different dirty values to write back concurrently).

1.1 Programmable Logic Devices

A **PLD** (*Programmable Logic Device*) is a way to implement custom computation hardware for a given application.

We said (in some course, don't remember which) that there are a few approaches to computation logic design:

- **Full custom IC**s: these obviously give the best performance and power consumption, but with very high NRE costs (masks, design). Designing a custom IC requires a long time-to-market, and is good only for demanding very high volume designs;
- **Gate array**s: these have very good performance and power, with high NRE costs (masks). We only need to build the higher layers masks, not the full IC like in full custom. Still, this has long time-to-market, and is good for high volume designs.

  Note that, altough **FPGA**s (*Field Programmable Gate Array*) have "gate array" in their name, they are usually considered PLDs. The good mental model is that gate arrays require additional IC routing after manufacturing, whereas PLDs are programmable after manufacturing, typically using SRAM configuration bits, flash, or antifuses;
- **Programmable Logic Device**s: good performance and power (compared to software), with low NRE costs (compared to a custom IC). There are no masks involved, and the user buys an already manufactured integrated circuit. This involves higher per-unit costs, so it's good for low volume designs and prototypes.

To summarize, a PLD is an integrated circuit (hardware device) that contains logic elements of various complexities. The interface allows the designer to configure and connect them in different ways, configurating the device after manufacturing. A PLD implements a hardware digital circuit, where the function of the circuit is configured by the user of the PLD. The masks do not depend on the implemented function, so the same device can be used for different applications. PLDs can be produced in large volumes, with NRE costs (for the PLD manufacturer) spread over many different customers.

PLDs can be configured to implement a logic circuit, either combinational or sequential. The final circuit is hardware, not software, but it is usually described using a high level "programming" language (VHDL, Verilog, SystemC, Chisel, HLS). Designers define internal logic functions, Interconnections and pin, using said language.

Technologies used to configure PLDs are:

- Fuse, anti-fuse (one time programmable, think OTP-ROMs, 7.1);
- Floating gate (reprogrammable, think EPROMs and EEPROMs, 7.2 and 7.2.5);
- SRAM (reprogrammable, reconfigurable at run-time, faster than floating gate technolog.

1.1.1 ROM based PLDs

We know from Shannon's theorem that memories (LUTs) and multiplexers can implement any combinational logic functions. For instance, a $2^m \times n$ can implement any set of n functions of m shared inputs.

The memory can be OTPROM, EPROM, E2PROM (permanent), SRAM (volatile). Definitely, it can't be ROM, otherwise we would make it full custom. Also not NAND Flash, because of the clunky sequential read access. Lastly, not DRAM, because of the complex access protocol (and active refreshes).

When used as a circuit, the memory is always read: it is written only before its use, to configure the circuit.